

I. Tipo de programación que se hará con Spotify

Debido a que Spotify Connect y su Embedded SDK están restringidos a socios de hardware certificados, el proyecto adopta una integración basada en la Spotify Web API. De este modo, se construye una bocina inteligente controlada desde el celular, donde Spotify se utiliza para la autenticación del usuario, la gestión de playlists y la obtención de metadatos, mientras que la reproducción de audio y el control del dispositivo se implementan mediante un sistema propio, manteniendo una experiencia similar a la de un equipo de sonido moderno conectado.

II. Comparación entre Spotify Connect y Spotify Web API

Spotify Connect

Función: convertir el dispositivo en un reproductor reconocido directamente por Spotify.

- [Spotify Connect](#)

Spotify Connect permite que un dispositivo aparezca dentro de la aplicación oficial de Spotify como si fuera una bocina inteligente, consola o Smart TV.

Ventajas

- El dispositivo aparece directamente en Spotify.
- Experiencia más profesional y parecida a productos comerciales.
- El audio puede reproducirse directamente desde el dispositivo.
- Compatible con control desde celular, PC o tablet.

Desventajas

- Mucho más complejo de implementar.
- Requiere descubrimiento en red local y sincronización constante.
- Necesita manejar streaming de audio en tiempo real.
- Spotify limita oficialmente esta función a partners comerciales y hardware autorizado.
- Consume más recursos de red y procesamiento.

Nivel de dificultad

Alto.

Recomendado para proyectos avanzados o productos comerciales.

Spotify Web API

Función: controlar la cuenta de Spotify del usuario mediante solicitudes web.

- [Spotify Web API](#)
La API oficial permite consultar canciones, controlar reproducción y acceder al estado de la cuenta mediante peticiones HTTP.
- Authorization Guide (OAuth 2.0)
Spotify utiliza OAuth 2.0 para autenticar usuarios y otorgar permisos seguros a las aplicaciones.

Scopes importantes

- `user-read-playback-state` → leer el estado actual de reproducción.
- `user-modify-playback-state` → pausar, cambiar canción o controlar volumen.
- `user-read-currently-playing` → obtener la canción actual.

Ventajas

- Mucho más fácil de implementar en ESP32 o Arduino.
- Solo necesita conexión a internet y solicitudes HTTP.
- Ideal para proyectos educativos o prototipos.
- Permite controlar Spotify desde botones, pantallas o sensores.
- Compatible con APIs REST y respuestas JSON.

Desventajas

- El dispositivo no aparece como reproductor Spotify Connect.
- Depende de que exista otro dispositivo reproduciendo música.
- Requiere autenticación OAuth 2.0 y manejo de tokens.

Nivel de dificultad

Medio.

Muy adecuado para proyectos escolares y sistemas embebidos.

Conclusión recomendada

Para un proyecto con ESP32, Arduino o microcontroladores, la mejor opción normalmente es **Spotify Web API**.

Justificación

- Es más sencilla de programar.
- Usa tecnologías comunes: Wi-Fi, HTTP y JSON.
- Funciona bien con librerías como:
 - [ArduinoJson](#)
 - [ArduinoHttpClient](#)
- Permite crear controles físicos reales:
 - Play/Pause
 - Siguiente canción
 - Volumen
 - Mostrar canción en pantalla OLED
 - LEDs reactivos

III. Arquitectura del hardware

Como el proyecto será programado en C++ usando ESP32, se utilizará una arquitectura basada en módulos separados para facilitar el control, audio, energía y conectividad.

Bloque de control

- ESP32
 - Será el cerebro principal del sistema.
 - Se encargará de:
 - Leer botones
 - Controlar LEDs
 - Manejar conexión Wi-Fi/Bluetooth

- Comunicarse con Spotify API
 - Enviar señales al módulo de audio
-

Bloque de alimentación

- Batería 18650 3.7V 5200mAh
Alimentará todo el sistema.
 - IP5306
Administrará la carga de la batería y protección eléctrica.
 - MT3608 Boost Converter
Elevará el voltaje de 3.7V a 5V para alimentar correctamente algunos módulos.
 - Switch de palanca
Permitirá encender y apagar el sistema.
-

Bloque de audio

- Bluetooth MP3 Decoder Module 747D
Recibirá y procesará el audio.
 - PAM8403
Amplificará la señal de audio.
 - Speaker
Reproducirá el sonido final.
-

Bloque de interacción

- Botón ×3
Se usarán para:
 - Play/Pause
 - Siguiente canción
 - Encendido de efectos o cambio de modo
- LED Rojo ×2
Funcionarán como indicadores de estado y alimentación.
- Cinta RGB
Permitirá efectos visuales sincronizados con la música o estados del sistema.

- Resistencia 220Ω
Protegerán LEDs y entradas electrónicas.
-

Bloque estructural

- Placa perforada
Se utilizará para montar el circuito final.
- Cable paralelo rojo/negro
Permitirá realizar las conexiones internas.
- Madera MDF
Servirá para construir la carcasa física del dispositivo.

IV. Librerías organizadas por función

Wi-Fi

Función: conectar el dispositivo a una red inalámbrica y mantener la conexión activa.

- WiFi (Arduino)
La biblioteca oficial de Arduino para módulos Wi-Fi. Permite conectarse a internet como cliente o crear un servidor local. Incluye funciones para escanear redes, reconectar automáticamente y gestionar direcciones IP.
 - [WiFi.h para ESP32](#)
Biblioteca usada en ESP32. Facilita la conexión a redes Wi-Fi, reconexión automática, modo Access Point y manejo de eventos de red.
-

HTTP / REST

Función: comunicarse con APIs y servicios web como Spotify, servidores IoT o páginas web.

- [ArduinoHttpClient](#)
Biblioteca para realizar solicitudes HTTP GET, POST y PUT desde Arduino. Muy útil para consumir APIs REST.

- [esp_http_client \(ESP-IDF\)](#)

Cliente HTTP oficial de Espressif para ESP32. Soporta HTTP y HTTPS, autenticación, envío de datos y descarga de contenido desde servidores web.

JSON

Función: interpretar y crear datos JSON enviados por APIs.

- [ArduinoJson](#)
Biblioteca especializada en serialización y deserialización JSON en C++. Permite leer respuestas de APIs y generar estructuras JSON de forma eficiente en microcontroladores con poca memoria.
-

LEDs direccionables y efectos de luz

Función: controlar tiras LED RGB y crear animaciones.

- [FastLED](#)
Biblioteca muy usada para LEDs direccionables como WS2812B o NeoPixel. Permite crear animaciones, efectos de color y control preciso del brillo.
 - [Adafruit NeoPixel](#)
Biblioteca optimizada para LEDs NeoPixel. Más sencilla para proyectos básicos de iluminación y efectos RGB.
-

Audio / Música

Función: reproducir audio, controlar volumen o manejar datos de sonido.

- [ESP8266Audio](#)
Biblioteca para reproducir archivos MP3, WAV y streams de audio en ESP32 y ESP8266.
 - [I2S Audio para ESP32](#)
Permite transmitir audio digital mediante el protocolo I2S hacia DACs, amplificadores o módulos de sonido.
-

Bluetooth

Función: comunicación inalámbrica de corto alcance con celulares, PCs u otros dispositivos.

- [BluetoothSerial \(ESP32\)](#)
Biblioteca para usar Bluetooth clásico en ESP32 como reemplazo de un puerto serial inalámbrico.
 - [NimBLE-Arduino](#)
Implementación ligera de Bluetooth Low Energy (BLE), optimizada para menor consumo de memoria y energía.
-

Sensores

Función: leer datos de temperatura, humedad, distancia, movimiento u otros sensores.

- [Adafruit Unified Sensor](#)
Proporciona una interfaz común para distintos sensores, facilitando el manejo de múltiples dispositivos.
 - [DHT Sensor Library](#)
Biblioteca específica para sensores DHT11 y DHT22 de temperatura y humedad.
-

Pantallas y visualización

Función: mostrar información en displays OLED, LCD o TFT.

- [Adafruit GFX Library](#)
Biblioteca gráfica base para dibujar texto, figuras e imágenes en múltiples tipos de pantallas.
 - [U8g2](#)
Biblioteca avanzada para displays OLED y LCD monocromáticos. Muy eficiente y compatible con muchos modelos.
-

Almacenamiento

Función: guardar configuraciones, archivos o datos del sistema.

- [SPIFFS](#)
Sistema de archivos para memoria flash interna en ESP32. Permite guardar archivos de configuración o páginas web.
- SD Library
Biblioteca oficial para leer y escribir archivos en tarjetas microSD.

Tiempo y sincronización

Función: manejar fecha, hora y sincronización con internet.

- [NTPClient](#)
Biblioteca para obtener la hora exacta desde servidores NTP usando internet.
-

MQTT / IoT

Función: comunicación eficiente entre dispositivos IoT y servidores.

- [PubSubClient](#)
Biblioteca MQTT ligera para enviar y recibir mensajes entre dispositivos IoT y brokers como Mosquitto o HiveMQ.

V. Comunicación del sistema con Spotify.

El sistema no implementa login local en la bocina. La autenticación del usuario se realiza externamente, desde un celular o computadora, mediante el sistema oficial de Spotify basado en OAuth 2.0. Una vez autorizada la cuenta, el dispositivo recibe permisos para interactuar con la Spotify Web API sin manejar credenciales sensibles.

La bocina incluye tres botones físicos que permiten controlar la reproducción de forma directa: avanzar a la siguiente canción, regresar a la canción anterior y pausar o reanudar la reproducción. Al presionar cada botón, el sistema envía el comando correspondiente a Spotify utilizando los permisos previamente otorgados, replicando el comportamiento de un equipo de sonido moderno con controles integrados.

VI. Pruebas para demostrar el correcto funcionamiento del sistema

Para validar que la bocina inteligente cumple con los objetivos del proyecto, se definirán una serie de **pruebas funcionales** que permitan comprobar el correcto comportamiento del sistema tanto a nivel de hardware como de software. Estas pruebas aseguran que el dispositivo opera como un equipo de sonido moderno, capaz de conectarse a la red, interactuar con Spotify y responder adecuadamente a las acciones del usuario.

Prueba de conexión Wi-Fi

Objetivo:

Verificar que el dispositivo puede conectarse correctamente a una red Wi-Fi disponible.

Procedimiento:

Al encenderse, el dispositivo intenta conectarse a la red Wi-Fi previamente configurada. El estado de conexión se indica mediante LEDs o mensajes en pantalla.

Resultado esperado:

La bocina se conecta exitosamente a la red Wi-Fi y muestra un estado visual que confirma la conexión activa. En caso de no encontrar la red, el sistema debe indicarlo claramente.

Prueba de reconexión automática

Objetivo:

Comprobar que el sistema es capaz de recuperarse ante una pérdida temporal de conexión.

Procedimiento:

Se interrumpe la conexión Wi-Fi (apagando el router o alejando el dispositivo del alcance) y luego se restablece.

Resultado esperado:

El sistema detecta la desconexión, entra en un estado de espera y se reconecta automáticamente sin necesidad de reiniciar el dispositivo ni repetir la configuración.

Prueba de autenticación con Spotify**Objetivo:**

Verificar que el sistema se autentica correctamente con Spotify sin utilizar login local.

Procedimiento:

El usuario inicia sesión en Spotify desde un celular o computadora asociados al proyecto. Se otorgan los permisos solicitados y se generan los tokens correspondientes.

Resultado esperado:

La autenticación se completa correctamente y el sistema puede realizar solicitudes a la Spotify Web API usando los permisos autorizados, sin manejar credenciales directamente en la bocina.

Prueba de descubrimiento del dispositivo

La prueba de descubrimiento del dispositivo evalúa la capacidad del sistema para detectar automáticamente la bocina dentro de la red local. Esta prueba confirma que el dispositivo se anuncia correctamente y puede ser identificado por la aplicación o interfaz de control del proyecto, permitiendo su uso sin configuraciones manuales. Aunque la bocina no aparece como un dispositivo de reproducción en Spotify, el descubrimiento

asegura que el sistema reconoce al equipo como disponible y listo para recibir comandos.

Resultado esperado:

El sistema reconoce la bocina como un dispositivo disponible para control, permitiendo enviar comandos de reproducción.

Pruebas de reproducción, pausa, siguiente y volumen

Objetivo:

Demostrar que el dispositivo responde correctamente a los comandos de control musical.

Procedimiento:

Se prueban los **tres botones físicos** del dispositivo:

- Botón de pausa / reproducción
- Botón de siguiente canción
- Botón de canción anterior

También se prueban los controles de volumen, ya sea mediante botones adicionales o comandos desde la interfaz de control.

Resultado esperado:

Cada botón ejecuta la acción correspondiente mediante la Spotify Web API. El audio responde de forma inmediata, y el estado del sistema se refleja mediante LEDs o pantalla.

Prueba de LEDs y pantalla

Objetivo:

Verificar que los elementos visuales informan correctamente el estado del sistema.

Procedimiento:

Se observan los LEDs y la pantalla durante los distintos estados del dispositivo: encendido, conexión Wi-Fi, autenticación, reproducción, pausa y error.

Resultado esperado:

Los LEDs y/o la pantalla muestran información coherente con el estado actual del sistema, facilitando la interacción del usuario sin necesidad del celular.

Prueba de manejo de errores**Objetivo:**

Evaluar cómo responde el sistema ante fallos o situaciones inesperadas.

Procedimiento:

Se simulan errores como:

- Fallo de conexión Wi-Fi
- Token de Spotify expirado
- Intento de control sin autenticación

Resultado esperado:

El sistema no se bloquea ni se reinicia de forma incorrecta. Los errores se indican mediante señales visuales y, cuando es posible, se recuperan automáticamente (por ejemplo, mediante refresh de token o reconexión a la red).

Conclusión de la etapa de pruebas

Estas pruebas permiten validar que el sistema cumple con su función principal y ayudan a identificar qué módulos deben implementarse con mayor prioridad, tanto en hardware como en software. Además, aseguran que el

proyecto no solo “funciona”, sino que ofrece una experiencia coherente, estable y propia de un dispositivo real.

VII. Módulos del software

Desde el inicio del diseño se plantea una arquitectura de software modular, evitando la implementación de un único archivo monolítico. El sistema se divide en módulos independientes, cada uno con una responsabilidad específica, como el manejo del estado del sistema, la conexión Wi-Fi, la comunicación con Spotify, la reproducción de audio, la interfaz de usuario, el manejo de errores y las rutinas de prueba. Esta separación permite un código más ordenado, mantenible y fácil de depurar, además de facilitar la integración de nuevas funciones sin afectar el funcionamiento general del sistema.

VIII. Limitaciones

Es importante destacar que la integración con Spotify presenta limitaciones que no son únicamente técnicas, sino también relacionadas con las políticas de uso de la plataforma, los mecanismos de autorización, el tipo de cuenta del usuario y los permisos otorgados mediante scopes. Spotify establece restricciones claras sobre cómo sus servicios pueden integrarse con dispositivos externos, lo que limita el acceso al streaming directo y a ciertas funcionalidades avanzadas disponibles solo para integraciones certificadas.

Asimismo, el sistema presenta limitaciones propias de su naturaleza física y de prototipo funcional, como las capacidades del hardware utilizado, los recursos disponibles para procesamiento, conectividad y manejo de audio, así como la necesidad de simplificar la interacción del usuario mediante botones, LEDs y pantallas de tamaño reducido. En consecuencia, el alcance del proyecto se define de manera realista, enfocándose en demostrar conceptos de conectividad, control de reproducción y diseño de sistemas integrados, sin prometer ni implementar características que excedan las restricciones técnicas, físicas o de uso impuestas por la plataforma y el hardware.